# Temporis:

## Una herramienta accesible para el control y la optimización de nuestro tiempo

Diseño de interfaces

Kevin Zamora Amela

## 1. Definición del Alcance de la Calidad

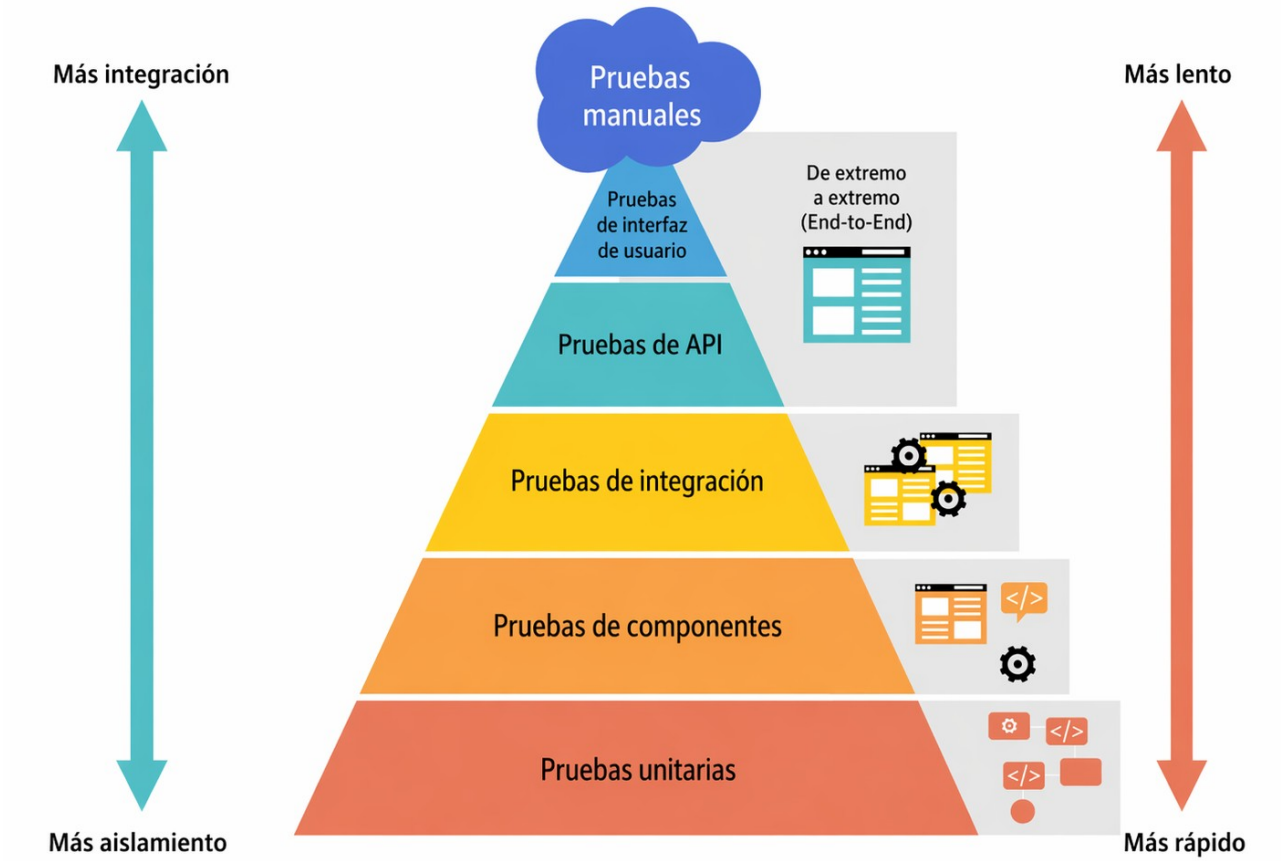
Para garantizar que **Temporis** sea una herramienta fiable y accesible, se han identificado los siguientes módulos y requisitos críticos como objetivos prioritarios del plan de calidad:

- **Módulos del Sistema:**
  - **Gestión de Autenticación:** Verificación de acceso seguro y protección de datos de usuario mediante *Firebase Auth*.
  - **Motor de Temporizadores (*Core*):** Exactitud en la cuenta atrás y persistencia de estados (activo/pausado).
  - **Sincronización *Cloud Firestore*:** Integridad de los datos al guardar y recuperar temporizadores y registros.
- **Requisitos No Funcionales Críticos:**
  - **Accesibilidad:** Verificación de contrastes y compatibilidad con lectores de pantalla (*TalkBack*).
  - **Rendimiento:** Tiempo de respuesta mínimo en la carga de estadísticas y noticias.

## 2. Diseño de la Estrategia de Pruebas

Se ha diseñado una estrategia multinivel que sigue la **pirámide de pruebas de software**, asegurando que los errores se detecten lo antes posible en el ciclo de vida.

*Pirámide de pruebas con pruebas de interfaz de usuario, pruebas de integración y pruebas unitarias*



## 2.1. Tabla Detallada de Estrategia de Pruebas

| Tipo de Prueba | Objetivo | Entorno y Herramientas | Responsable |
|---|---|---|---|
| **Unitarias** | Validar la lógica aislada de las clases (*User*, *Timer*, *TimeRegister*). | Android Studio / JUnit & Mockito | **Desarrollador** |
| **Integración** | Verificar la comunicación correcta entre la *App* y *Firebase* (CRUD). | Emulador Android / *Firebase Emulator* | **Desarrollador** |
| **Sistema (UI)** | Simular el flujo completo del usuario: *Login* -> Crear *Timer* -> Ejecutar. | Dispositivo Físico / *Espresso Framework* | **QA / Desarrollador** ( Dada la naturaleza individual del proyecto, el autor asumirá el rol de *QA Tester*, aplicando una separación de mentalidad entre la fase de desarrollo y la fase de "testeo" para garantizar la objetividad. ) |
| **Aceptación** | Validar que el diseño final coincide con los prototipos definidos en la tarea "Tr_13". | *Beta Test* (Usuario final) | **Usuario / Analista** |
| **Seguridad** | Comprobar que las *Security Rules* de *Firestore* impiden acceso no autorizado. | Consola de *Firebase* / Pruebas de inyección | **Desarrollador** |

## 3. Secuencia de Ejecución Lógica

La ejecución de las pruebas no es aleatoria, sino que sigue un orden de dependencia técnica:

1. **Fase 1: Verificación de Código (Unitarias):** Se ejecutan cada vez que se modifica una función lógica en Java/Kotlin. Es la base de la estabilidad.
2. **Fase 2: Conectividad (Integración):** Una vez que la lógica es sólida, probamos que los datos fluyen hacia la nube sin errores de red o de formato.
3. **Fase 3: Flujos de Usuario (Sistema):** Se comprueba la interfaz de usuario en su totalidad para detectar errores de navegación o de visualización (especialmente en el modo oscuro).
4. **Fase 4: Validación de Accesibilidad:** Se realizan pruebas específicas con herramientas de diagnóstico de Google para asegurar que el diseño es inclusivo.
5. **Fase 5: Cierre (Aceptación):** Revisión final para confirmar que el producto cumple con los objetivos y el alcance definidos en la "Tr_09" (Apartado:Planificación del Ciclo de Vida (Gantt)).